



MINISTRY OF EDUCATION, CULTURE AND RESEARCH OF THE
REPUBLIC OF MOLDOVA

Technical University of Moldova Faculty of Computers, Informatics and
Microelectronics Department of Software and Automation Engineering

Postoronca Dumitru FAF-233

Report

Laboratory work n.7

on Embedded Systems

Checked by:

A. Martiniuc, *university assistant*
DISA, FCIM, UTM

Chişinău – 2026

1 Purpose of laboratory work

The purpose of this laboratory work is to implement an **actuator control system** on the ESP32 microcontroller using FreeRTOS for concurrent task execution. The system controls two actuators – an analog actuator (DC motor via L298N PWM) and a binary actuator (relay module) – with commands received over serial, a signal conditioning pipeline, and real-time reporting on an OLED display. This is Variant C (100%), requiring both analog and binary actuator control.

Hardware note: Due to the unavailability of a DC motor during the laboratory session, the analog actuator output (GPIO 18, ENA pin of the L298N) is connected to an LED with a $220\ \Omega$ current-limiting resistor instead of a real motor. The LED brightness varies proportionally with the PWM duty cycle, allowing the ramp-up, ramp-down, and speed control behavior to be visually verified. The L298N direction pins (GPIO 16, GPIO 17) are wired and driven correctly – swapping in a real motor requires no code changes whatsoever.

1.1 Objectives of the laboratory work

Laboratory work has the following objectives that should be achieved:

- Implementing a serial command interface for receiving actuator commands (`S<0-100>`, `DF`, `DR`, `X`, `RON`, `ROFF`, `?`) at a 50 ms polling period
- Implementing analog actuator control through a four-stage command conditioning pipeline: saturation (clamping to 0–100 %), a median filter (window size 5) for impulsive command noise removal, an exponential moving average (EMA, $\alpha = 0.3$) for smoothing, and a ramping stage (2 % per 50 ms cycle) that limits the rate of change applied to the motor
- Controlling a DC motor (L298N, GPIO 16/17/18) via PWM (LEDC 5 kHz, 8-bit) with direction control (forward/reverse) and an emergency stop that bypasses the ramp and resets all filters
- Controlling a single-channel relay module (GPIO 19, active-HIGH) as a binary actuator with ON/OFF target tracking
- Evaluating alert conditions (`OVERLOAD` $\geq 95\%$, `LIMIT_REACHED` at 0 % or 100 %) and driving LED indicators accordingly

- Displaying a structured report at 500 ms on an OLED 128×64 display showing motor state, direction, target/conditioned/current speed, relay state, and alert; emitting a serial plotter line every 500 ms
- Structuring the firmware into four independent FreeRTOS tasks (Command, Conditioning, Actuator, Reporter) communicating through a binary semaphore and a mutex
- Organizing the codebase into three hardware-abstraction layers: MCAL, ECAL, and SRV

1.2 Problem definition

The application is structured in four distinct FreeRTOS tasks:

1. **Task 1 – Command (TaskCommand):** Non-blocking serial read every 50 ms. Parses commands, validates ranges, stores target speed, direction, relay target, and emergency stop flag in shared `ActuatorData`. Signals the conditioning task via a binary semaphore after each valid speed command.
2. **Task 2 – Signal Conditioning (TaskConditioning):** Runs every 50 ms, blocked on the binary semaphore with timeout. Applies a four-stage pipeline:
 - (a) **Saturation** – clamps the target to 0–100 %
 - (b) **Median filter** (window = 5) – removes impulsive command noise
 - (c) **Exponential Moving Average** ($\alpha = 0.3$) – smooths fluctuations: $y_n = \alpha \cdot x_n + (1 - \alpha) \cdot y_{n-1}$
 - (d) **Ramping** – increments `currentSpeed` toward `conditionedSpeed` by at most 2 % per cycle (≈ 2.5 s full ramp), manages motor state transitions (IDLE / RAMP_UP / RUNNING / RAMP_DOWN)

Evaluates alerts and drives green/red LED indicators.

3. **Task 3 – Actuator (TaskActuator):** Runs every 50 ms. Reads `currentSpeed` and `direction` from `ActuatorData`, maps 0–100 % to PWM duty 0–255 and applies it via the L298N LEDC driver. Reads `relayTarget` and applies it to the relay driver when it differs from the current relay state.

4. **Task 4 – Reporter (TaskReporter):** Runs every 500 ms using `vTaskDelayUntil`. Updates the OLED 128×64 with all actuator state fields, emits a serial plotter line in `>name:value` format, and every 5 seconds prints a full structured report to STDIO.

2 Domain Analysis

2.1 Application Context and Utilized Technologies

The primary objective of this laboratory work is to implement an **actuator control system** for both an analog and a binary actuator using **FreeRTOS** on the ESP32 microcontroller. The application demonstrates a complete actuator pipeline: serial command reception, multi-stage command conditioning (saturation, median filtering, EMA smoothing, ramping), alert detection, and structured reporting on an OLED display and via serial.

The **analog actuator** is a DC motor connected through an **L298N** dual H-bridge driver. GPIO 16 and GPIO 17 are used as direction pins (IN1/IN2), while GPIO 18 is the PWM speed pin (ENA). The ESP32's LEDC peripheral generates a 5 kHz PWM signal with 8-bit resolution (0–255 duty cycle). In this implementation a **PWM-driven LED** is connected to GPIO 18 in place of the motor, so that the ramping and speed control can be visually verified without motor hardware.

The **binary actuator** is a **1-channel active-HIGH relay module** on GPIO 19. The relay is energized or de-energized by a single digital output, and its state is tracked in shared data to avoid redundant writes.

The command conditioning stage implements four classical techniques applied in sequence:

- **Saturation (clamping):** The raw command value is clamped to the valid actuator range of 0–100 %. Out-of-range commands are rejected with an error message and clamped, preventing impossible targets from entering the pipeline.
- **Median filter** (window = 5): A sliding window stores the five most recent saturated targets. Each cycle the window is sorted and the median is selected. This non-linear filter is highly effective at removing isolated impulsive noise (e.g., a momentary wrong command) without distorting the underlying command trend.
- **Exponential Moving Average (EMA):** A first-order IIR low-pass filter with coefficient $\alpha = 0.3$ is applied to the median-filtered output:

$$y_n = \alpha \cdot x_n + (1 - \alpha) \cdot y_{n-1}$$

This provides weighted smoothing where recent commands contribute 30% and the accumulated history contributes 70%, reducing abrupt step changes.

- **Ramping:** The `currentSpeed` variable is incremented or decremented toward `conditionedSpeed` by a fixed step of 2 % per 50 ms cycle (≈ 2.5 s for a full 0–100 % ramp). This limits the rate of change delivered to the motor, protecting it from sudden current surges. An **emergency stop** (command `X`) bypasses the ramp entirely and sets `currentSpeed` to zero immediately, also resetting all filter state.

After the full pipeline, alert conditions are evaluated:

- **OVERLOAD** – conditioned speed ≥ 95 %
- **LIMIT_REACHED** – conditioned speed at 0 % or 100 %

Task synchronization uses two FreeRTOS primitives:

- A **binary semaphore** (`xSemNewCommand`) given by `TaskCommand` after each valid speed command and taken by `TaskConditioning`, ensuring the conditioning pipeline runs on new data
- A **mutex** (`xActuatorMutex`) protecting all shared actuator state fields against concurrent access by the four tasks

2.2 Hardware and Software Components

Component	Description & Role
ESP32 DevKit	Dual-core microcontroller running FreeRTOS. Hosts all four application tasks and provides GPIO, LEDC PWM, I2C, and UART peripherals.
L298N H-bridge	Dual H-bridge motor driver. IN1=GPIO 16, IN2=GPIO 17 (direction), ENA=GPIO 18 (PWM speed). Logic supply 5 V; motor supply up to 46 V.
LED (PWM test)	Connected to GPIO 18 (ENA) via a 220 Ω resistor. Simulates motor speed response; brightness proportional to duty cycle (0–255).
Relay module	1-channel active-HIGH relay on GPIO 19. Energized by RON, de-energized by ROFF. VCC from 5 V rail.
Green LED	Connected to GPIO 25. ON when no alert is active (system OK).
Red LED	Connected to GPIO 26. ON when alert is active (OVERLOAD or LIMIT_REACHED).
OLED 128×64 SSD1306	I2C bus (SDA=GPIO 21, SCL=GPIO 22), address 0x3C. Displays motor state, direction, target/conditioned/current speed, relay state, and alert at 500 ms.
220 Ω Resistors	Current-limiting resistors for each LED.
Serial-to-USB Interface	UART at 115200 baud for command input and serial plotter output (>name:value format).
FreeRTOS	Built into the ESP32 Arduino core. Provides preemptive task scheduling, binary semaphore, and mutex primitives.
PlatformIO	Build system used for compilation, upload, and serial monitoring.
Serial Plotter (VS Code)	VS Code extension for real-time plotting of >name:value data. Displays target, conditioned, and current speed traces side by side to visualize the ramp and filter effects.
Breadboard	Prototyping base for connecting all components without soldering.

2.3 System Architecture and Solution Justification

The system is organized into three hardware-abstraction layers following the MCAL / ECAL / SRV pattern:

1. **MCAL (Microcontroller Abstraction Layer):** Contains only `GpioDriver`, which wraps `pinMode`, `digitalWrite`, and `digitalRead`. All direct Arduino hardware calls are isolated here.
2. **ECAL (Electronic Component Abstraction Layer):** Contains component drivers: `MotorDriver` (L298N LEDC PWM + direction pins), `RelayDriver` (single GPIO on/off), `LedDriver` (two status LEDs), `OledDisplay` (Adafruit SSD1306), and `SerialIO` (UART init). Each driver header declares its pin assignments for easy circuit reference.
3. **SRV (Service Layer):** Contains `ActuatorData` (shared data store with mutex/semaphore and all configurable parameters) and the four FreeRTOS task implementations: `TaskCommand`, `TaskConditioning`, `TaskActuator`, and `TaskReporter`.

The data flow proceeds as follows:

1. **TaskCommand** polls the serial port every 50 ms. On a valid speed command it stores `targetSpeed` in `ActuatorData` and gives `xSemNewCommand`. Direction, relay, and emergency stop commands are applied immediately without signaling.
2. **TaskConditioning** runs every 50 ms (semaphore with timeout). It applies the four-stage pipeline (saturation → median → EMA → ramp), stores intermediate values, updates `motorState`, evaluates alerts, and drives the LEDs.
3. **TaskActuator** runs every 50 ms. It reads `currentSpeed` and `direction`, maps speed to a 0–255 duty cycle, and calls `MotorDriver_SetDirection + MotorDriver_SetSpeed` (or `MotorDriver_Stop` when speed < 1%). It also compares `relayTarget` with `relayState` and calls `RelayDriver_On/Off` on a change.
4. **TaskReporter** runs every 500 ms. It reads all shared values, updates the OLED, emits a serial plotter line, and every 5 seconds prints a full structured report to STDIO.

2.4 Relevant Case Study

A real-world application of command conditioning with ramping and relay control is **industrial conveyor belt systems**. Speed set-points from an operator console are subject to operator mistakes (an accidentally typed 150 %) and communication glitches (isolated burst errors). A saturation stage rejects out-of-range set-points, a median filter absorbs isolated bursts, and a weighted moving average smooths repeated small adjustments. The ramping stage then ensures the belt motor accelerates and decelerates within its mechanical limits, preventing belt slip and mechanical stress. A relay control simultaneously switches auxiliary equipment (e.g., a diverter gate) on or off based on the operator command – the exact two-actuator pattern implemented in this laboratory work.

3 Design

3.1 Architectural sketch

Below is the architectural sketch of the components structured in the diagram that highlights the software and hardware components.

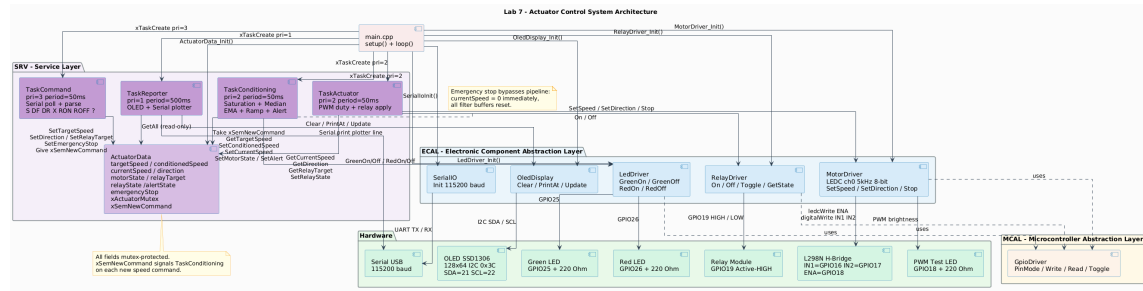


Figure 1: Architectural sketch of the project

This is the description of each of the elements of the diagram:

1. **ESP32 DevKit microcontroller** – central processing unit running FreeRTOS and executing all four application tasks preemptively
2. **L298N motor driver (IN1=GPIO 16, IN2=GPIO 17, ENA=GPIO 18)** – H-bridge driver controlled by TaskActuator; direction set via IN1/IN2, PWM speed via ENA using LEDC (channel 0, 5 kHz, 8-bit); LED on ENA simulates motor during testing
3. **Relay module (GPIO 19)** – active-HIGH single-channel relay controlled by TaskActuator; state transitions tracked in ActuatorData
4. **Green LED (GPIO 25)** – ON when no alert is active
5. **Red LED (GPIO 26)** – ON when OVERLOAD or LIMIT_REACHED alert is active
6. **OLED 128×64 SSD1306 (SDA=21, SCL=22, addr 0x3C)** – primary display updated by TaskReporter every 500 ms showing motor state, direction, target/conditioned/current speed, relay state, and alert

7. **ActuatorData module** – shared data store protected by `xActuatorMutex`; holds `targetSpeed`, `conditionedSpeed`, `currentSpeed`, `direction`, `motorState`, `relayTarget`, `relayState`, `alertState`, `emergencyStop` flag, and all configurable pipeline parameters
8. **TaskCommand** – FreeRTOS task (priority 3) polling serial at 50 ms; parses and validates commands; gives `xSemNewCommand` on speed commands
9. **TaskConditioning** – FreeRTOS task (priority 2) running every 50 ms; applies saturation, median, EMA, and ramp; evaluates alerts; updates LEDs
10. **TaskActuator** – FreeRTOS task (priority 2) running every 50 ms; maps `currentSpeed` to PWM duty and applies to `MotorDriver`; applies relay state changes via `RelayDriver`
11. **TaskReporter** – FreeRTOS task (priority 1) running every 500 ms; updates OLED and prints STDIO reports
12. **GpioDriver (MCAL)** – wraps all `digitalRead/digitalWrite` calls; only layer that touches Arduino hardware APIs directly

3.2 Electrical sketch

The circuit diagram outlines the setup for this project, showing the connections between the ESP32, the L298N motor driver, the relay module, two status LEDs, the OLED SSD1306 display, and the computer for serial communication.

Each LED (green and red) is connected to its respective GPIO pin through a $220\ \Omega$ current-limiting resistor to GND. The PWM test LED on GPIO 18 (ENA) also uses a $220\ \Omega$ resistor to GND. The L298N logic supply is connected to the ESP32 5 V rail, with IN1 and IN2 driven by GPIO 16 and GPIO 17. The relay module signal pin is connected to GPIO 19, with VCC from the 5 V rail. The OLED SSD1306 shares the I2C bus (SDA=GPIO 21, SCL=GPIO 22) at address 0x3C.

Figure 2 details the connections and components that make up the system. This visual documentation is crucial for replicating the setup or diagnosing issues, as it clearly shows how each component is integrated into the overall design.

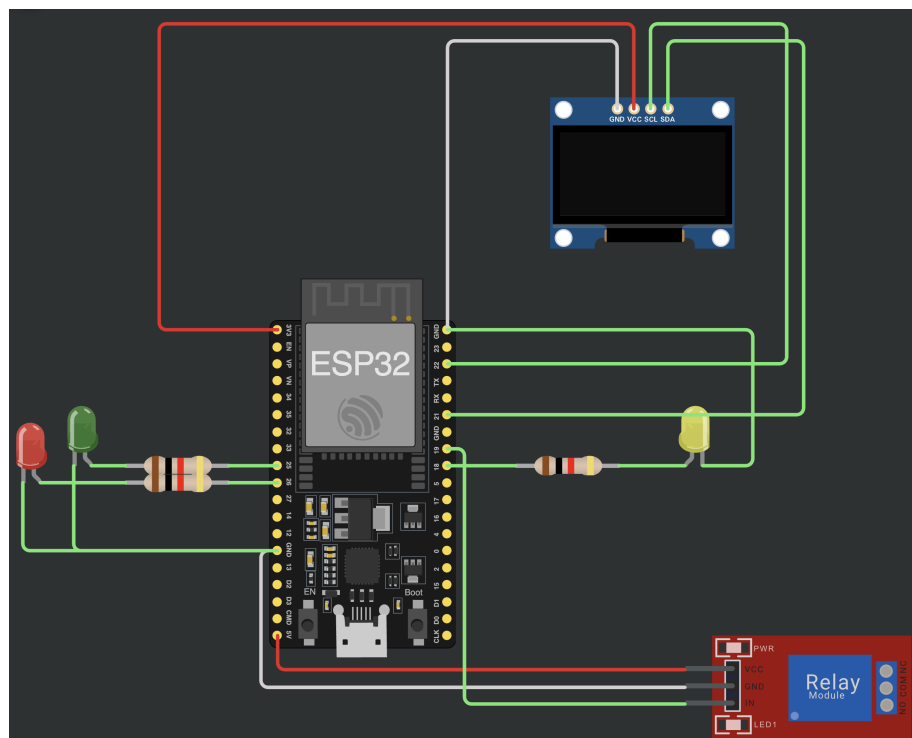


Figure 2: Circuit sketched on wokwi

3.3 Flow chart

The Figure 3 illustrates the FreeRTOS task interaction flow. TaskCommand polls the serial port every 50ms and gives the semaphore on valid speed commands. TaskConditioning wakes on the semaphore (or after a 50 ms timeout), applies the four-stage pipeline, and updates `currentSpeed` and `motorState`. TaskActuator reads `currentSpeed` and applies it to the motor driver every 50ms. TaskReporter runs every 500ms independently and reads shared data to refresh the OLED and serial output.

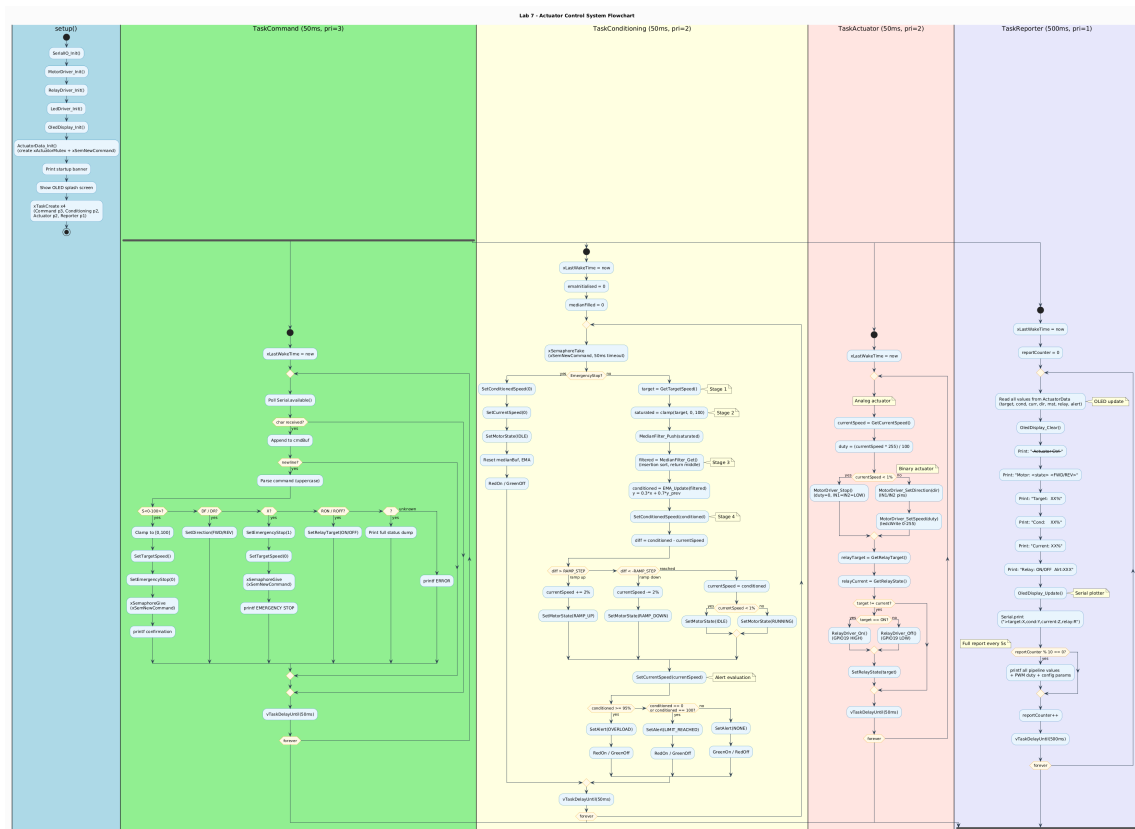


Figure 3: Flow chart of the algorithm

3.4 Code design

Code is developed following the principle of separating the interface from the implementation, by creating for each header file `.h` a respective `.cpp` file. The project uses **PlatformIO** with three library directories:

1. **MCAL layer** (`lib/MCAL/GpioDriver`) – hardware-specific code that wraps `Arduino.h` GPIO calls. All direct hardware access is encapsulated here.
2. **ECAL layer** (`lib/ECAL/`) – electronic component drivers: `MotorDriver` (L298N PWM via LEDC + direction pins, GPIO 16/17/18), `RelayDriver` (GPIO 19 on/off), `LedDriver` (two status LEDs, GPIO 25/26), `OledDisplay` (Adafruit SSD1306 128×64), and `SerialIO` (UART initialization). Each driver header declares its connection pins for easy circuit reference.
3. **SRV layer** (`lib/SRV/`) – the `ActuatorData` shared-state module and the four FreeRTOS task implementations.

The `MotorDriver` initializes the LEDC peripheral and provides a clean API that is identical for a real motor or an LED:

```
1 void MotorDriver_Init(void)
2 {
3     GpioDriver_PinMode(MOTOR_IN1_PIN, OUTPUT);
4     GpioDriver_PinMode(MOTOR_IN2_PIN, OUTPUT);
5     GpioDriver_Write(MOTOR_IN1_PIN, LOW);
6     GpioDriver_Write(MOTOR_IN2_PIN, LOW);
7     ledcSetup(MOTOR_LEDC_CHANNEL, MOTOR_LEDC_FREQ, MOTOR_LEDC_RES);
8     ledcAttachPin(MOTOR_ENA_PIN, MOTOR_LEDC_CHANNEL);
9     ledcWrite(MOTOR_LEDC_CHANNEL, 0);
10 }
11
12 void MotorDriver_SetSpeed(uint8_t duty)    // 0-255
13 {
14     currentDuty = duty;
15     ledcWrite(MOTOR_LEDC_CHANNEL, duty);
16 }
17
18 void MotorDriver_SetDirection(uint8_t dir) // 0=forward, 1=reverse
19 {
20     if (dir == 0) {
21         GpioDriver_Write(MOTOR_IN1_PIN, HIGH);
22         GpioDriver_Write(MOTOR_IN2_PIN, LOW);
23     } else {
```

```

24     GpioDriver_Write(MOTOR_IN1_PIN, LOW);
25     GpioDriver_Write(MOTOR_IN2_PIN, HIGH);
26 }
27 }

```

The `ActuatorData` module centralizes all shared data behind mutex-protected getters and setters, and owns the FreeRTOS synchronization primitives:

```

1 // Declared in ActuatorData.h, initialized in ActuatorData_Init()
2 extern SemaphoreHandle_t xSemNewCommand; // binary semaphore: new
   speed command
3 extern SemaphoreHandle_t xActuatorMutex; // mutex: protects all
   shared actuator fields

```

The signal conditioning pipeline in `TaskConditioning` applies all four stages then manages motor state transitions:

```

1 /* 1. Saturation */
2 float saturated = Saturate(target); // clamp to [0, 100]
3
4 /* 2. Median filter (window=5) */
5 MedianFilter_Push(saturated);
6 float filtered = MedianFilter_Get();
7
8 /* 3. EMA smoothing (alpha=0.3) */
9 float conditioned = EMA_Update(filtered);
10 ActuatorData_SetConditionedSpeed(conditioned);
11
12 /* 4. Ramping (2% per cycle) */
13 float diff = conditioned - currentSpeed;
14 if (diff > RAMP_STEP) { currentSpeed += RAMP_STEP; state =
   MOTOR_RAMP_UP; }
15 else if (diff < -RAMP_STEP) { currentSpeed -= RAMP_STEP; state =
   MOTOR_RAMP_DOWN; }
16 else { currentSpeed = conditioned;
17       state = (currentSpeed < 1.0f) ? MOTOR_IDLE : MOTOR_RUNNING
   ; }
18 ActuatorData_SetCurrentSpeed(currentSpeed);

```

`TaskActuator` maps the conditioned speed to a PWM duty cycle and applies it:

```

1 float currentSpeed = ActuatorData_GetCurrentSpeed();

```

```
2 uint8_t duty = (uint8_t)((currentSpeed * 255.0f) / 100.0f);
3
4 if (currentSpeed < 1.0f) {
5     MotorDriver_Stop();
6 } else {
7     MotorDriver_SetDirection((uint8_t)dir);
8     MotorDriver_SetSpeed(duty);
9 }
```

TaskReporter uses vTaskDelayUntil for drift-corrected 500 ms periodicity and outputs data in the >name:value format compatible with the VS Code Serial Plotter extension:

```
1 // Serial Plotter line (every 500 ms)
2 Serial.print(">");
3 Serial.print("target:"); Serial.print(target);
4 Serial.print(",cond:");   Serial.print(cond, 1);
5 Serial.print(",current:"); Serial.print(curr, 1);
6 Serial.print(",relay:");   Serial.print(relay == RELAY_ON ? 1 : 0);
7 Serial.println();
8
9 // Full structured report (every 5 s)
10 if ((reportCounter++ % FULL_REPORT_EVERY) == 0) {
11     printf("Target speed   : %d%\n", target);
12     printf("Conditioned    : %.1f%% (median=%d, EMA alpha=%.2f)\n",
13           cond, MEDIAN_WINDOW, EMA_ALPHA);
14     printf("Current speed : %.1f%% (ramp step=%d/cyc)\n", curr,
15           RAMP_STEP);
16     printf("Motor state   : %s\n", MotorStateName(mst));
17     printf("Relay        : %s\n", relay == RELAY_ON ? "ON" : "OFF");
18     ;
19     printf("Alert         : %s\n", AlertName(alert));
```


4 Results

For running the application the following commands were used for compiling, uploading the project and monitoring the port to observe the periodic reports.

```
# list connected boards and their ports
pio device list
# compile and upload to ESP32
pio run --target upload
# open serial monitor at 115200 baud
pio device monitor --baud 115200
```

Below is the startup banner and sample serial output of the system:

```
=====
  Lab 7 - Actuator Control System (Var C)
=====
Analog   : L298 PWM on GPIO18 (LED test)
           IN1=GPIO16  IN2=GPIO17
Binary   : Relay on GPIO19 (active-HIGH)
Pipeline: Sat -> Median(5) -> EMA(a=0.30) -> Ramp(2/cyc)
Overload: >= 95%
Commands: S<0-100> DF DR X RON ROFF ?
=====

[CMD] Speed target -> 50%
>target:50,cond:8.0,current:4.0,relay:0
>target:50,cond:18.4,current:10.0,relay:0
>target:50,cond:26.9,current:16.0,relay:0

===== ACTUATOR REPORT (t=5000 ms) =====
Target speed   : 50%
Conditioned    : 44.2% (median=5, EMA alpha=0.30)
Current speed  : 40.0% (ramp step=2/cyc)
Direction     : FORWARD
Motor state    : RAMP_UP
Relay          : OFF
Alert         : NONE
PWM duty      : 102/255
```

```
=====

[CMD] Speed target -> 100%
>target:100,cond:95.0,current:80.0,relay:0
[CMD] Relay -> ON
>target:100,cond:98.0,current:82.0,relay:1

===== ACTUATOR REPORT (t=10000 ms) =====
Target speed   : 100%
Conditioned    : 99.1% (median=5, EMA alpha=0.30)
Current speed  : 88.0% (ramp step=2/cyc)
Direction     : FORWARD
Motor state    : RAMP_UP
Relay         : ON
Alert         : OVERLOAD
PWM duty      : 224/255
=====

[CMD] EMERGENCY STOP
>target:0,cond:0.0,current:0.0,relay:1
```

Hardware note: A DC motor was not available during the laboratory session. The analog actuator output (GPIO 18, ENA) is therefore connected to an LED with a $220\ \Omega$ resistor, which serves as a visual substitute: its brightness is proportional to the PWM duty cycle and clearly demonstrates the ramping, smoothing, and speed control behavior. The L298N direction pins (GPIO 16, GPIO 17) are wired and driven correctly; replacing the LED with a real motor requires no code changes.

The system behavior is as follows:

- At startup, the OLED shows -- Actuator Ctrl -- / Lab7 Variant C / Starting... for 1 second while all modules initialize.
- After a speed command (e.g., S50), `currentSpeed` increments by 2% every 50ms, with the OLED showing the motor state transition from IDLE → RAMP_UP → RUNNING. The LED on GPIO 18 brightens gradually, visually confirming the ramp behavior in place of a real motor.
- The **conditioned speed** lags the target due to the median and EMA filters. Rapid command changes (e.g., S10 then S90 then S30) are smoothed, and the ramp ensures the LED brightness changes at a controlled rate.

- Sending **S100** causes the conditioned speed to reach $\geq 95\%$, triggering **OVERLOAD** alert: the red LED turns on, the green LED turns off, and the OLED shows **Alrt:OVERLD**.
- Sending **S0** causes the motor state to transition to RAMP_DOWN and eventually IDLE. Sending **X** instead causes an **immediate** stop (no ramping), re-setting all filter state.
- **DR** followed by **S50** swaps the IN1/IN2 direction pins – the motor would reverse; the LED brightness is unchanged.
- **R0N** energizes the relay (GPIO 19 HIGH) and the OLED shows **Relay: ON**. **R0FF** de-energizes it.
- **?** prints a full status dump to the serial terminal at any time.
- The serial plotter line **>target:X,cond:Y,current:Z,relay:R** is emitted every 500 ms. When visualized in the VS Code Serial Plotter extension, the three speed traces show the filtering and ramp effects clearly: **target** steps instantly, **cond** follows with smoothing lag, and **current** ramps at a constant slope.
- Entering **S150** is rejected with an error message and clamped to 100 % before entering the pipeline.

Below is the visual representation of the circuit:

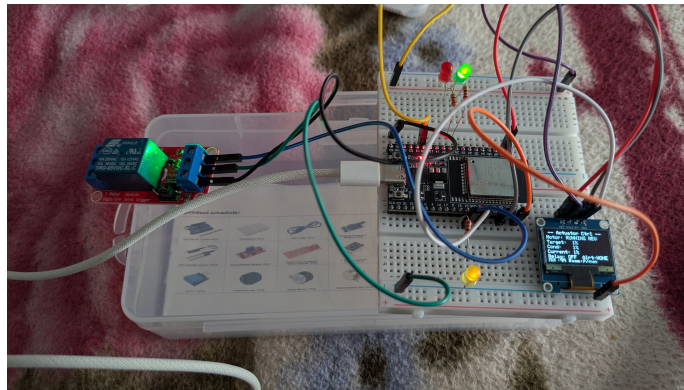


Figure 4: Circuit in working state

Figure 4 shows the circuit with the OLED and LEDs connected.

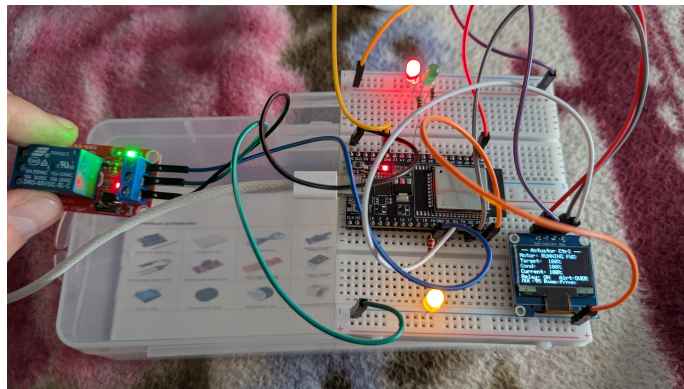


Figure 5: Circuit in working state

Figure 5 shows how the system will look when all the peripherals are ramped to maximum (relay on, Full power on the motor)

5 Conclusion

Completion of this laboratory work has provided practical understanding of actuator control techniques for embedded systems, including analog PWM motor control, binary relay control, and a multi-stage command conditioning pipeline, while reinforcing the use of FreeRTOS synchronization primitives for safe inter-task communication.

This seventh laboratory work covered the complete actuator control pipeline from serial command reception through multi-stage command conditioning to hardware actuation and structured reporting. The key takeaways are:

1. Implementing a **serial command interface** in FreeRTOS context – the command task polls the serial port every 50 ms, parses case-insensitive commands, validates ranges, and signals the conditioning task via semaphore, keeping the command parsing decoupled from the conditioning logic.
2. Applying a **four-stage command conditioning pipeline**:
 - **Saturation** clamps the raw command to the valid actuator range (0–100 %), rejecting physically impossible targets at the input boundary.
 - **Median filter** (window = 5) removes impulsive command noise – isolated wrong commands are eliminated without distorting the underlying command trend, as the median is inherently robust to up to $\lfloor n/2 \rfloor$ outliers in the window.
 - **Exponential Moving Average** ($\alpha = 0.3$) provides weighted smoothing where recent commands contribute 30% and the accumulated history contributes 70%, reducing abrupt step changes visible to the actuator.
 - **Ramping** (2 % per 50 ms cycle) limits the rate of change of `currentSpeed`, protecting the motor from sudden current surges and implementing smooth acceleration and deceleration (≈ 2.5 s for a full 0–100 % ramp). The motor state machine (IDLE / RAMP_UP / RUNNING / RAMP_DOWN) makes the ramp phase clearly visible on the OLED and in the serial plotter.
3. Implementing an **emergency stop** (X) that bypasses the conditioning pipeline entirely – `currentSpeed` is set to zero immediately, all filter state (median buffer, EMA accumulator) is reset, and the motor stops without any ramp. This demonstrates the importance of having a safety override path independent of the normal control loop.

4. Controlling an **analog actuator** (DC motor via L298N) using the ESP32 LEDC peripheral: channel 0, 5 kHz, 8-bit resolution, with direction pins managed separately. The MotorDriver API (Init / SetSpeed / SetDirection / Stop / GetDuty) is structured so that swapping the test LED for a real motor requires zero code changes.
5. Controlling a **binary actuator** (relay) with a simple ON/OFF API and state tracking in shared data. The relay is only written when its target differs from the current state, avoiding redundant GPIO writes.
6. Evaluating **alert conditions** ($\text{OVERLOAD} \geq 95\%$, `LIMIT_REACHED` at boundaries) on the conditioned speed and driving LED indicators accordingly, providing clear visual feedback about the actuator's operating region.
7. Structuring the firmware into four distinct **FreeRTOS tasks** with well-defined roles and priorities: Command (priority 3) ensures serial input is never missed, Conditioning and Actuator (priority 2) run the control loop, and Reporter (priority 1) provides periodic human-readable output without interfering with the control loop.
8. Using a **mutex-protected shared data store** (`ActuatorData`) as the single source of truth for all actuator state – target, conditioned, and current speed; direction; motor state; relay states; and alerts – preventing data races when four tasks read or write concurrently.
9. Organizing the project into **three hardware-abstraction layers** (MCAL, ECAL, SRV) with PlatformIO, keeping all hardware-specific code in the MCAL layer and making the service layer fully portable across different MCU platforms.

This laboratory work demonstrates how a pipeline of classical command processing techniques – saturation, median filtering, exponential moving averaging, and ramping – combined with FreeRTOS synchronization primitives, transforms raw serial commands into smooth, safe actuator control suitable for both analog (PWM motor) and binary (relay) actuators in an embedded system.

References

- [1] Espressif Systems. (2023). *ESP32 Series Datasheet*. Retrieved from https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf
- [2] Espressif Systems. (2024). *ESP32 LEDC (LED Control) Peripheral*. Retrieved from <https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/peripherals/ledc.html>
- [3] STMicroelectronics. (2000). *L298 Dual Full-Bridge Driver Datasheet*. Retrieved from <https://www.st.com/resource/en/datasheet/l298.pdf>
- [4] Adafruit Industries. (2024). *Adafruit SSD1306 OLED Driver Library*. Retrieved from https://github.com/adafruit/Adafruit_SSD1306
- [5] Real Time Engineers Ltd. (2024). *FreeRTOS API Reference*. Retrieved from <https://www.freertos.org/a00106.html>
- [6] Barry, R. (2016). *Mastering the FreeRTOS Real Time Kernel – A Hands-On Tutorial Guide*. Real Time Engineers Ltd. Retrieved from https://www.freertos.org/Documentation/RTOS_book.html
- [7] Barr, M. (2006). *Programming Embedded Systems: With C and GNU Development Tools*. O'Reilly Media, Inc.
- [8] Smith, S. W. (1997). *The Scientist and Engineer's Guide to Digital Signal Processing*. California Technical Publishing. Retrieved from <https://www.dspguide.com/>
- [9] Oppenheim, A. V., Willsky, A. S., & Nawab, S. H. (1996). *Signals and Systems* (2nd ed.). Prentice Hall.

A Source Code

A.1 Main Entry Point

```

1  #include "SerialIO.h"
2  #include "MotorDriver.h"
3  #include "RelayDriver.h"
4  #include "LedDriver.h"
5  #include "OledDisplay.h"
6  #include "ActuatorData.h"
7  #include "TaskCommand.h"
8  #include "TaskConditioning.h"
9  #include "TaskActuator.h"
10 #include "TaskReporter.h"
11
12 void setup()
13 {
14     /*          Driver / ECAL initialisation          */
15     SerialIoInit();
16     MotorDriver_Init();
17     RelayDriver_Init();
18     LedDriver_Init();
19     OledDisplay_Init();
20
21     /*          Service layer initialisation          */
22     ActuatorData_Init();
23
24     printf("\n=====");
25     printf("  Lab 7 - Actuator Control System (Var C)\n");
26     printf("=====");
27     printf("Analog   : L298 PWM on GPIO%d (LED test)\n",
28     MOTOR_ENA_PIN);
29     printf("          IN1=GPIO%d  IN2=GPIO%d\n", MOTOR_IN1_PIN,
30     MOTOR_IN2_PIN);
31     printf("Binary   : Relay on GPIO%d (active-HIGH)\n", RELAY_PIN);
32     printf("Pipeline: Sat -> Median(%d) -> EMA(a=%.2f) -> Ramp(%d/
33     cyc)\n",
34     MEDIAN_WINDOW, EMA_ALPHA, RAMP_STEP);
35     printf("Overload: >= %d%%\n", OVERLOAD_THRESHOLD);
36     printf("Commands: S<0-100> DF DR X RON ROFF ?\n");
37     printf("=====");
38
39     OledDisplay_Clear();
40     OledDisplay_SetTextSize(1);
41     OledDisplay_PrintAt(4, 0, "-- Actuator Ctrl --");

```



```
39   OledDisplay_PrintAt(0, 32, "Starting...");
40   OledDisplay_Update();
41
42   vTaskDelay(pdMS_TO_TICKS(1000));
43
44   /*      FreeRTOS tasks      */
45   xTaskCreate(TaskCommand_Task,      "Command",      2048, NULL,
46   3, NULL);
47   xTaskCreate(TaskConditioning_Task, "Conditioning", 2048, NULL,
48   2, NULL);
49   xTaskCreate(TaskActuator_Task,     "Actuator",     2048, NULL,
50   2, NULL);
51   xTaskCreate(TaskReporter_Task,     "Reporter",     4096, NULL,
52   1, NULL);
53 }
54
55 void loop()
56 {
57     vTaskDelay(portMAX_DELAY);
58 }
```

Listing 1: main.cpp - Application Entry Point

A.2 MCAL Layer

```
1 #pragma once
2 #include <Arduino.h>
3
4 void GpioDriver_PinMode(uint8_t pin, uint8_t mode);
5 void GpioDriver_Write(uint8_t pin, uint8_t val);
6 uint8_t GpioDriver_Read(uint8_t pin);
7 void GpioDriver_Toggle(uint8_t pin);
```

Listing 2: GpioDriver.h - GPIO Abstraction Header

```
1 #include "GpioDriver.h"
2
3 void GpioDriver_PinMode(uint8_t pin, uint8_t mode)
4 {
5     pinMode(pin, mode);
6 }
7
8 void GpioDriver_Write(uint8_t pin, uint8_t val)
9 {
10    digitalWrite(pin, val);
```

```
11 }
12
13 uint8_t GpioDriver_Read(uint8_t pin)
14 {
15     return digitalRead(pin);
16 }
17
18 void GpioDriver_Toggle(uint8_t pin)
19 {
20     digitalWrite(pin, !digitalRead(pin));
21 }
```

Listing 3: GpioDriver.cpp - GPIO Abstraction Implementation

A.3 ECAL Layer – Actuator Drivers and Peripherals

```
1 #pragma once
2 #include <stdint.h>
3
4 #define MOTOR_IN1_PIN 16
5 #define MOTOR_IN2_PIN 17
6 #define MOTOR_ENA_PIN 18 /* PWM speed (LEDC) */
7
8 #define MOTOR_LEDC_CHANNEL 0
9 #define MOTOR_LEDC_FREQ 5000 /* 5 kHz */
10 #define MOTOR_LEDC_RES 8 /* 8-bit: 0-255 */
11
12 void MotorDriver_Init(void);
13 void MotorDriver_SetSpeed(uint8_t duty); /* 0-255 */
14 void MotorDriver_SetDirection(uint8_t dir); /* 0=forward, 1=
    reverse */
15 void MotorDriver_Stop(void);
16 uint8_t MotorDriver_GetDuty(void);
```

Listing 4: MotorDriver.h - L298N PWM Driver Header

```
1 #include "MotorDriver.h"
2 #include "GpioDriver.h"
3 #include <Arduino.h>
4
5 static uint8_t currentDuty = 0;
6
7 void MotorDriver_Init(void)
8 {
9     GpioDriver_PinMode(MOTOR_IN1_PIN, OUTPUT);
```

```

10  GpioDriver_PinMode(MOTOR_IN2_PIN, OUTPUT);
11  GpioDriver_Write(MOTOR_IN1_PIN, LOW);
12  GpioDriver_Write(MOTOR_IN2_PIN, LOW);
13
14  ledcSetup(MOTOR_LEDC_CHANNEL, MOTOR_LEDC_FREQ, MOTOR_LEDC_RES);
15  ledcAttachPin(MOTOR_ENA_PIN, MOTOR_LEDC_CHANNEL);
16  ledcWrite(MOTOR_LEDC_CHANNEL, 0);
17  currentDuty = 0;
18 }
19
20 void MotorDriver_SetSpeed(uint8_t duty)
21 {
22     currentDuty = duty;
23     ledcWrite(MOTOR_LEDC_CHANNEL, duty);
24 }
25
26 void MotorDriver_SetDirection(uint8_t dir)
27 {
28     if (dir == 0) {
29         /* Forward: IN1=HIGH, IN2=LOW */
30         GpioDriver_Write(MOTOR_IN1_PIN, HIGH);
31         GpioDriver_Write(MOTOR_IN2_PIN, LOW);
32     } else {
33         /* Reverse: IN1=LOW, IN2=HIGH */
34         GpioDriver_Write(MOTOR_IN1_PIN, LOW);
35         GpioDriver_Write(MOTOR_IN2_PIN, HIGH);
36     }
37 }
38
39 void MotorDriver_Stop(void)
40 {
41     currentDuty = 0;
42     ledcWrite(MOTOR_LEDC_CHANNEL, 0);
43     GpioDriver_Write(MOTOR_IN1_PIN, LOW);
44     GpioDriver_Write(MOTOR_IN2_PIN, LOW);
45 }
46
47 uint8_t MotorDriver_GetDuty(void)
48 {
49     return currentDuty;
50 }

```

Listing 5: MotorDriver.cpp - L298N PWM Driver Implementation

```

1  #pragma once
2  #include <stdint.h>

```

```
3
4 /* --- Wiring --- */
5 #define RELAY_PIN 19 /* Active-HIGH 1-channel relay module */
6
7 void RelayDriver_Init(void);
8 void RelayDriver_On(void);
9 void RelayDriver_Off(void);
10 void RelayDriver_Toggle(void);
11 uint8_t RelayDriver_GetState(void); /* 0=off, 1=on */
```

Listing 6: RelayDriver.h - Relay Driver Header

```
1 #include "RelayDriver.h"
2 #include "GpioDriver.h"
3
4 static uint8_t relayState = 0;
5
6 void RelayDriver_Init(void)
7 {
8     GpioDriver_PinMode(RELAY_PIN, OUTPUT);
9     GpioDriver_Write(RELAY_PIN, LOW);
10    relayState = 0;
11 }
12
13 void RelayDriver_On(void)
14 {
15     GpioDriver_Write(RELAY_PIN, HIGH);
16     relayState = 1;
17 }
18
19 void RelayDriver_Off(void)
20 {
21     GpioDriver_Write(RELAY_PIN, LOW);
22     relayState = 0;
23 }
24
25 void RelayDriver_Toggle(void)
26 {
27     if (relayState) {
28         RelayDriver_Off();
29     } else {
30         RelayDriver_On();
31     }
32 }
33
34 uint8_t RelayDriver_GetState(void)
```

```
35 {  
36     return relayState;  
37 }
```

Listing 7: RelayDriver.cpp - Relay Driver Implementation

```
1 #pragma once  
2  
3 /* --- Wiring --- */  
4 #define LED_GREEN_PIN 25  
5 #define LED_RED_PIN 26  
6  
7 void LedDriver_Init(void);  
8 void LedDriver_GreenOn(void);  
9 void LedDriver_GreenOff(void);  
10 void LedDriver_RedOn(void);  
11 void LedDriver_RedOff(void);
```

Listing 8: LedDriver.h - LED Driver Header

```
1 #include "LedDriver.h"  
2 #include "GpioDriver.h"  
3  
4 void LedDriver_Init(void)  
5 {  
6     GpioDriver_PinMode(LED_GREEN_PIN, OUTPUT);  
7     GpioDriver_PinMode(LED_RED_PIN, OUTPUT);  
8     GpioDriver_Write(LED_GREEN_PIN, LOW);  
9     GpioDriver_Write(LED_RED_PIN, LOW);  
10 }  
11  
12 void LedDriver_GreenOn(void) { GpioDriver_Write(LED_GREEN_PIN, HIGH  
    ); }  
13 void LedDriver_GreenOff(void) { GpioDriver_Write(LED_GREEN_PIN, LOW)  
    ; }  
14 void LedDriver_RedOn(void) { GpioDriver_Write(LED_RED_PIN, HIGH  
    ); }  
15 void LedDriver_RedOff(void) { GpioDriver_Write(LED_RED_PIN, LOW)  
    ; }
```

Listing 9: LedDriver.cpp - LED Driver Implementation

```
1 #pragma once  
2 #include <stdint.h>  
3  
4 /* --- Wiring (I2C, shared bus with LCD) --- */
```

```
5 #define OLED_WIDTH      128
6 #define OLED_HEIGHT     64
7 #define OLED_I2C_ADDR   0x3C
8 #define OLED_SDA_PIN    21  /* ESP32 default I2C SDA */
9 #define OLED_SCL_PIN    22  /* ESP32 default I2C SCL */
10
11 void OledDisplay_Init(void);
12 void OledDisplay_Clear(void);
13 void OledDisplay_SetCursor(uint8_t x, uint8_t y);
14 void OledDisplay_SetTextSize(uint8_t size);
15 void OledDisplay_Print(const char *str);
16 void OledDisplay_PrintAt(uint8_t x, uint8_t y, const char *str);
17 void OledDisplay_Update(void);
```

Listing 10: OledDisplay.h - OLED SSD1306 Driver Header

```
1 #include "OledDisplay.h"
2 #include <Wire.h>
3 #include <Adafruit_GFX.h>
4 #include <Adafruit_SSD1306.h>
5
6 static Adafruit_SSD1306 oled(OLED_WIDTH, OLED_HEIGHT, &Wire, -1);
7
8 void OledDisplay_Init(void)
9 {
10     oled.begin(SSD1306_SWITCHCAPVCC, OLED_I2C_ADDR);
11     oled.clearDisplay();
12     oled.setTextColor(SSD1306_WHITE);
13     oled.setTextSize(1);
14     oled.display();
15 }
16
17 void OledDisplay_Clear(void)
18 {
19     oled.clearDisplay();
20 }
21
22 void OledDisplay_SetCursor(uint8_t x, uint8_t y)
23 {
24     oled.setCursor(x, y);
25 }
26
27 void OledDisplay_SetTextSize(uint8_t size)
28 {
29     oled.setTextSize(size);
30 }
```

```
31
32 void OledDisplay_Print(const char *str)
33 {
34     oled.print(str);
35 }
36
37 void OledDisplay_PrintAt(uint8_t x, uint8_t y, const char *str)
38 {
39     oled.setCursor(x, y);
40     oled.print(str);
41 }
42
43 void OledDisplay_Update(void)
44 {
45     oled.display();
46 }
```

Listing 11: OledDisplay.cpp - OLED SSD1306 Driver Implementation

```
1 #pragma once
2
3 #define SERIAL_BAUD 115200
4
5 void SerialIoInit(void);
```

Listing 12: SerialIO.h - Serial Initialization Header

```
1 #include "SerialIO.h"
2 #include <Arduino.h>
3
4 void SerialIoInit(void)
5 {
6     Serial.begin(SERIAL_BAUD);
7     delay(2000);
8 }
```

Listing 13: SerialIO.cpp - Serial Initialization Implementation

A.4 SRV Layer – Shared Data

```
1 #pragma once
2 #include <freertos/FreeRTOS.h>
3 #include <freertos/semphr.h>
4 #include <stdint.h>
5
```

```

6  /*      Task periods (ms)      */
7  #define COMMAND_PERIOD_MS      50
8  #define CONDITIONING_PERIOD_MS  50
9  #define ACTUATOR_PERIOD_MS      50
10 #define REPORTER_PERIOD_MS      500
11 #define FULL_REPORT_EVERY      10  /* 10 x 500ms = 5s */
12
13 /*      Speed limits (%)      */
14 #define SPEED_MIN      0
15 #define SPEED_MAX      100
16
17 /*      Signal conditioning      */
18 #define MEDIAN_WINDOW      5
19 #define EMA_ALPHA      0.3f
20 #define RAMP_STEP      2  /* % per 50ms cycle -> ~2.5s full
    ramp */
21 #define OVERLOAD_THRESHOLD  95  /* % */
22
23 /*      Direction      */
24 typedef enum { DIR_FORWARD = 0, DIR_REVERSE = 1 } Direction_t;
25
26 /*      Motor state machine      */
27 typedef enum {
28     MOTOR_IDLE = 0,
29     MOTOR_RAMP_UP,
30     MOTOR_RUNNING,
31     MOTOR_RAMP_DOWN
32 } MotorState_t;
33
34 /*      Relay state      */
35 typedef enum { RELAY_OFF = 0, RELAY_ON = 1 } RelayState_t;
36
37 /*      Alert state      */
38 typedef enum {
39     ALERT_NONE = 0,
40     ALERT_LIMIT_REACHED,
41     ALERT_OVERLOAD
42 } AlertState_t;
43
44 /*      Synchronisation      */
45 extern SemaphoreHandle_t xSemNewCommand;
46 extern SemaphoreHandle_t xActuatorMutex;
47
48 /*      Init      */
49 void ActuatorData_Init(void);
50

```



```

51  /*      Target speed (0-100%, raw from command)      */
52  void      ActuatorData_SetTargetSpeed(int16_t val);
53  int16_t   ActuatorData_GetTargetSpeed(void);
54
55  /*      Conditioned speed (after median + EMA)      */
56  void      ActuatorData_SetConditionedSpeed(float val);
57  float     ActuatorData_GetConditionedSpeed(void);
58
59  /*      Current speed (after ramping, actually applied)      */
60  void      ActuatorData_SetCurrentSpeed(float val);
61  float     ActuatorData_GetCurrentSpeed(void);
62
63  /*      Direction      */
64  void      ActuatorData_SetDirection(Direction_t dir);
65  Direction_t ActuatorData_GetDirection(void);
66
67  /*      Motor state      */
68  void      ActuatorData_SetMotorState(MotorState_t st);
69  MotorState_t ActuatorData_GetMotorState(void);
70
71  /*      Relay      */
72  void      ActuatorData_SetRelayTarget(RelayState_t st);
73  RelayState_t ActuatorData_GetRelayTarget(void);
74  void      ActuatorData_SetRelayState(RelayState_t st);
75  RelayState_t ActuatorData_GetRelayState(void);
76
77  /*      Alert      */
78  void      ActuatorData_SetAlert(AlertState_t a);
79  AlertState_t ActuatorData_GetAlert(void);
80
81  /*      Emergency stop flag      */
82  void      ActuatorData_SetEmergencyStop(uint8_t flag);
83  uint8_t   ActuatorData_GetEmergencyStop(void);

```

Listing 14: ActuatorData.h - Shared Actuator Data Header

```

1  #include "ActuatorData.h"
2
3  SemaphoreHandle_t xSemNewCommand = NULL;
4  SemaphoreHandle_t xActuatorMutex = NULL;
5
6  static int16_t      gTargetSpeed      = 0;
7  static float        gConditionedSpeed = 0.0f;
8  static float        gCurrentSpeed     = 0.0f;
9  static Direction_t  gDirection        = DIR_FORWARD;
10 static MotorState_t  gMotorState       = MOTOR_IDLE;

```

```
11 static RelayState_t gRelayTarget      = RELAY_OFF;
12 static RelayState_t gRelayState       = RELAY_OFF;
13 static AlertState_t gAlert            = ALERT_NONE;
14 static uint8_t      gEmergencyStop    = 0;
15
16 void ActuatorData_Init(void)
17 {
18     xSemNewCommand = xSemaphoreCreateBinary();
19     xActuatorMutex = xSemaphoreCreateMutex();
20 }
21
22 /*      Target speed      */
23 void ActuatorData_SetTargetSpeed(int16_t val)
24 {
25     xSemaphoreTake(xActuatorMutex, portMAX_DELAY);
26     gTargetSpeed = val;
27     xSemaphoreGive(xActuatorMutex);
28 }
29
30 int16_t ActuatorData_GetTargetSpeed(void)
31 {
32     xSemaphoreTake(xActuatorMutex, portMAX_DELAY);
33     int16_t val = gTargetSpeed;
34     xSemaphoreGive(xActuatorMutex);
35     return val;
36 }
37
38 /*      Conditioned speed      */
39 void ActuatorData_SetConditionedSpeed(float val)
40 {
41     xSemaphoreTake(xActuatorMutex, portMAX_DELAY);
42     gConditionedSpeed = val;
43     xSemaphoreGive(xActuatorMutex);
44 }
45
46 float ActuatorData_GetConditionedSpeed(void)
47 {
48     xSemaphoreTake(xActuatorMutex, portMAX_DELAY);
49     float val = gConditionedSpeed;
50     xSemaphoreGive(xActuatorMutex);
51     return val;
52 }
53
54 /*      Current speed      */
55 void ActuatorData_SetCurrentSpeed(float val)
56 {
```

```
57     xSemaphoreTake(xActuatorMutex, portMAX_DELAY);
58     gCurrentSpeed = val;
59     xSemaphoreGive(xActuatorMutex);
60 }
61
62 float ActuatorData_GetCurrentSpeed(void)
63 {
64     xSemaphoreTake(xActuatorMutex, portMAX_DELAY);
65     float val = gCurrentSpeed;
66     xSemaphoreGive(xActuatorMutex);
67     return val;
68 }
69
70 /*          Direction          */
71 void ActuatorData_SetDirection(Direction_t dir)
72 {
73     xSemaphoreTake(xActuatorMutex, portMAX_DELAY);
74     gDirection = dir;
75     xSemaphoreGive(xActuatorMutex);
76 }
77
78 Direction_t ActuatorData_GetDirection(void)
79 {
80     xSemaphoreTake(xActuatorMutex, portMAX_DELAY);
81     Direction_t val = gDirection;
82     xSemaphoreGive(xActuatorMutex);
83     return val;
84 }
85
86 /*          Motor state          */
87 void ActuatorData_SetMotorState(MotorState_t st)
88 {
89     xSemaphoreTake(xActuatorMutex, portMAX_DELAY);
90     gMotorState = st;
91     xSemaphoreGive(xActuatorMutex);
92 }
93
94 MotorState_t ActuatorData_GetMotorState(void)
95 {
96     xSemaphoreTake(xActuatorMutex, portMAX_DELAY);
97     MotorState_t val = gMotorState;
98     xSemaphoreGive(xActuatorMutex);
99     return val;
100 }
101
102 /*          Relay target          */
```

```
103 void ActuatorData_SetRelayTarget(RelayState_t st)
104 {
105     xSemaphoreTake(xActuatorMutex, portMAX_DELAY);
106     gRelayTarget = st;
107     xSemaphoreGive(xActuatorMutex);
108 }
109
110 RelayState_t ActuatorData_GetRelayTarget(void)
111 {
112     xSemaphoreTake(xActuatorMutex, portMAX_DELAY);
113     RelayState_t val = gRelayTarget;
114     xSemaphoreGive(xActuatorMutex);
115     return val;
116 }
117
118 /*      Relay state      */
119 void ActuatorData_SetRelayState(RelayState_t st)
120 {
121     xSemaphoreTake(xActuatorMutex, portMAX_DELAY);
122     gRelayState = st;
123     xSemaphoreGive(xActuatorMutex);
124 }
125
126 RelayState_t ActuatorData_GetRelayState(void)
127 {
128     xSemaphoreTake(xActuatorMutex, portMAX_DELAY);
129     RelayState_t val = gRelayState;
130     xSemaphoreGive(xActuatorMutex);
131     return val;
132 }
133
134 /*      Alert      */
135 void ActuatorData_SetAlert(AlertState_t a)
136 {
137     xSemaphoreTake(xActuatorMutex, portMAX_DELAY);
138     gAlert = a;
139     xSemaphoreGive(xActuatorMutex);
140 }
141
142 AlertState_t ActuatorData_GetAlert(void)
143 {
144     xSemaphoreTake(xActuatorMutex, portMAX_DELAY);
145     AlertState_t val = gAlert;
146     xSemaphoreGive(xActuatorMutex);
147     return val;
148 }
```

```

149
150  /*      Emergency stop      */
151  void ActuatorData_SetEmergencyStop(uint8_t flag)
152  {
153      xSemaphoreTake(xActuatorMutex, portMAX_DELAY);
154      gEmergencyStop = flag;
155      xSemaphoreGive(xActuatorMutex);
156  }
157
158  uint8_t ActuatorData_GetEmergencyStop(void)
159  {
160      xSemaphoreTake(xActuatorMutex, portMAX_DELAY);
161      uint8_t val = gEmergencyStop;
162      xSemaphoreGive(xActuatorMutex);
163      return val;
164  }

```

Listing 15: ActuatorData.cpp - Shared Actuator Data Implementation

A.5 SRV Layer – FreeRTOS Tasks

```

1  #pragma once
2  #include <freertos/FreeRTOS.h>
3  #include <freertos/task.h>
4
5  void TaskCommand_Task(void *pvParameters);

```

Listing 16: TaskCommand.h - Command Task Header

```

1  #include "TaskCommand.h"
2  #include "ActuatorData.h"
3  #include <Arduino.h>
4
5  #define CMD_BUF_SIZE 16
6
7  static char cmdBuf[CMD_BUF_SIZE];
8  static uint8_t cmdIdx = 0;
9
10 static void ProcessCommand(const char *cmd)
11 {
12     /* Make uppercase copy for case-insensitive parsing */
13     char upper[CMD_BUF_SIZE];
14     uint8_t i = 0;
15     while (cmd[i] && i < CMD_BUF_SIZE - 1) {
16         upper[i] = toupper(cmd[i]);

```

```

17         i++;
18     }
19     upper[i] = '\0';
20
21     if (upper[0] == 'S' && upper[1] >= '0' && upper[1] <= '9') {
22         /* S<0-100> : Set speed */
23         int speed = atoi(&upper[1]);
24         if (speed < SPEED_MIN || speed > SPEED_MAX) {
25             printf("[CMD] ERROR: speed %d out of range [%d-%d],
26             clamping\n",
27                     speed, SPEED_MIN, SPEED_MAX);
28             if (speed < SPEED_MIN) speed = SPEED_MIN;
29             if (speed > SPEED_MAX) speed = SPEED_MAX;
30         }
31         ActuatorData_SetEmergencyStop(0);
32         ActuatorData_SetTargetSpeed((int16_t)speed);
33         xSemaphoreGive(xSemNewCommand);
34         printf("[CMD] Speed target -> %d%%\n", speed);
35     } else if (strcmp(upper, "DF") == 0) {
36         ActuatorData_SetDirection(DIR_FORWARD);
37         printf("[CMD] Direction -> FORWARD\n");
38     } else if (strcmp(upper, "DR") == 0) {
39         ActuatorData_SetDirection(DIR_REVERSE);
40         printf("[CMD] Direction -> REVERSE\n");
41     } else if (strcmp(upper, "X") == 0) {
42         ActuatorData_SetEmergencyStop(1);
43         ActuatorData_SetTargetSpeed(0);
44         xSemaphoreGive(xSemNewCommand);
45         printf("[CMD] EMERGENCY STOP\n");
46     } else if (strcmp(upper, "RON") == 0) {
47         ActuatorData_SetRelayTarget(RELAY_ON);
48         printf("[CMD] Relay -> ON\n");
49     } else if (strcmp(upper, "ROFF") == 0) {
50         ActuatorData_SetRelayTarget(RELAY_OFF);
51         printf("[CMD] Relay -> OFF\n");
52     } else if (strcmp(upper, "?") == 0) {
53         /* Status query - print full report */
54         float target = (float)ActuatorData_GetTargetSpeed();
55         float cond   = ActuatorData_GetConditionedSpeed();
56         float curr    = ActuatorData_GetCurrentSpeed();

```

```

62     Direction_t dir    = ActuatorData_GetDirection();
63     MotorState_t mst    = ActuatorData_GetMotorState();
64     RelayState_t rst    = ActuatorData_GetRelayState();
65     AlertState_t alert  = ActuatorData_GetAlert();
66
67     printf("\n===== STATUS QUERY =====\n");
68     printf("Target speed : %.0f%%\n", target);
69     printf("Conditioned   : %.1f%%\n", cond);
70     printf("Current       : %.1f%%\n", curr);
71     printf("Direction    : %s\n", dir == DIR_FORWARD ? "FORWARD"
: "REVERSE");
72     printf("Motor state  : %s\n",
73           mst == MOTOR_IDLE ? "IDLE" :
74           mst == MOTOR_RAMP_UP ? "RAMP_UP" :
75           mst == MOTOR_RUNNING ? "RUNNING" : "RAMP_DOWN");
76     printf("Relay       : %s\n", rst == RELAY_ON ? "ON" : "OFF"
);
77     printf("Alert       : %s\n",
78           alert == ALERT_NONE ? "NONE" :
79           alert == ALERT_LIMIT_REACHED ? "LIMIT_REACHED" : "
OVERLOAD");
80     printf("=====\n\n");
81
82     } else {
83         printf("[CMD] ERROR: unknown command '%s'\n", cmd);
84         printf("[CMD] Usage: S<0-100> DF DR X RON ROFF ?\n");
85     }
86 }
87
88 void TaskCommand_Task(void *pvParameters)
89 {
90     TickType_t xLastWakeTime = xTaskGetTickCount();
91
92     for (;;) {
93         /* Non-blocking serial read */
94         while (Serial.available() > 0) {
95             char c = (char)Serial.read();
96
97             if (c == '\n' || c == '\r') {
98                 if (cmdIdx > 0) {
99                     cmdBuf[cmdIdx] = '\0';
100                     ProcessCommand(cmdBuf);
101                     cmdIdx = 0;
102                 }
103             } else if (cmdIdx < CMD_BUF_SIZE - 1) {
104                 cmdBuf[cmdIdx++] = c;

```

```

105         }
106     }
107
108     vTaskDelayUntil(&xLastWakeTime, pdMS_TO_TICKS(
109         COMMAND_PERIOD_MS));
110 }

```

Listing 17: TaskCommand.cpp - Command Task Implementation

```

1 #pragma once
2 #include <freertos/FreeRTOS.h>
3 #include <freertos/task.h>
4
5 void TaskConditioning_Task(void *pvParameters);

```

Listing 18: TaskConditioning.h - Conditioning Task Header

```

1 #include "TaskConditioning.h"
2 #include "ActuatorData.h"
3 #include "LedDriver.h"
4
5 /*          Saturation (clamp to valid range)          */
6 static float Saturate(float val)
7 {
8     if (val < (float)SPEED_MIN) return (float)SPEED_MIN;
9     if (val > (float)SPEED_MAX) return (float)SPEED_MAX;
10    return val;
11 }
12
13 /*          Median filter (removes impulsive command noise)          */
14 static float medianBuf[MEDIAN_WINDOW];
15 static uint8_t medianIdx = 0;
16 static uint8_t medianFilled = 0;
17
18 static void MedianFilter_Push(float val)
19 {
20     medianBuf[medianIdx] = val;
21     medianIdx = (medianIdx + 1) % MEDIAN_WINDOW;
22     if (medianFilled < MEDIAN_WINDOW) medianFilled++;
23 }
24
25 static float MedianFilter_Get(void)
26 {
27     if (medianFilled == 0) return 0.0f;
28 }

```



```
29     float sorted[MEDIAN_WINDOW];
30     uint8_t n = medianFilled;
31     for (uint8_t i = 0; i < n; i++) sorted[i] = medianBuf[i];
32
33     /* Insertion sort (n <= 5) */
34     for (uint8_t i = 1; i < n; i++) {
35         float key = sorted[i];
36         int j = i - 1;
37         while (j >= 0 && sorted[j] > key) {
38             sorted[j + 1] = sorted[j];
39             j--;
40         }
41         sorted[j + 1] = key;
42     }
43     return sorted[n / 2];
44 }
45
46 /*      Exponential Moving Average      */
47 static float emaValue = 0.0f;
48 static int   emaInitialised = 0;
49
50 static float EMA_Update(float newVal)
51 {
52     if (!emaInitialised) {
53         emaValue = newVal;
54         emaInitialised = 1;
55     } else {
56         emaValue = EMA_ALPHA * newVal + (1.0f - EMA_ALPHA) *
emaValue;
57     }
58     return emaValue;
59 }
60
61 /*      Alert evaluation      */
62 static AlertState_t EvaluateAlert(float conditioned)
63 {
64     if (conditioned >= (float)OVERLOAD_THRESHOLD)
65         return ALERT_OVERLOAD;
66     if (conditioned <= (float)SPEED_MIN || conditioned >= (float)
SPEED_MAX)
67         return ALERT_LIMIT_REACHED;
68     return ALERT_NONE;
69 }
70
71 /*
72 * TaskConditioning - signal-conditioning pipeline for actuator
```

```

    commands:
73 *   1. Saturation (clamp to 0-100%)
74 *   2. Median filter (window=5, removes impulsive noise)
75 *   3. EMA smoothing (alpha=0.3)
76 *   4. Ramping (increment currentSpeed toward conditioned by
    RAMP_STEP)
77 *   5. Alert evaluation
78 */
79 void TaskConditioning_Task(void *pvParameters)
80 {
81     TickType_t xLastWakeTime = xTaskGetTickCount();
82
83     for (;;) {
84         /* Wait for new command or timeout */
85         xSemaphoreTake(xSemNewCommand, pdMS_TO_TICKS(
CONDITIONING_PERIOD_MS));
86
87         /* Read target */
88         float target = (float)ActuatorData_GetTargetSpeed();
89
90         /* Check emergency stop */
91         if (ActuatorData_GetEmergencyStop()) {
92             /* Immediate stop: bypass all filtering */
93             ActuatorData_SetConditionedSpeed(0.0f);
94             ActuatorData_SetCurrentSpeed(0.0f);
95             ActuatorData_SetMotorState(MOTOR_IDLE);
96             ActuatorData_SetAlert(ALERT_NONE);
97
98             /* Reset filters */
99             medianIdx = 0;
100             medianFilled = 0;
101             emaInitialised = 0;
102             emaValue = 0.0f;
103
104             LedDriver_RedOn();
105             LedDriver_GreenOff();
106
107             vTaskDelayUntil(&xLastWakeTime, pdMS_TO_TICKS(
CONDITIONING_PERIOD_MS));
108             continue;
109         }
110
111         /* 1. Saturation */
112         float saturated = Saturate(target);
113
114         /* 2. Median filter */

```

```

115     MedianFilter_Push(saturated);
116     float filtered = MedianFilter_Get();
117
118     /* 3. EMA smoothing */
119     float conditioned = EMA_Update(filtered);
120     ActuatorData_SetConditionedSpeed(conditioned);
121
122     /* 4. Ramping */
123     float currentSpeed = ActuatorData_GetCurrentSpeed();
124     float diff = conditioned - currentSpeed;
125
126     if (diff > (float)RAMP_STEP) {
127         currentSpeed += (float)RAMP_STEP;
128         ActuatorData_SetMotorState(MOTOR_RAMP_UP);
129     } else if (diff < -(float)RAMP_STEP) {
130         currentSpeed -= (float)RAMP_STEP;
131         ActuatorData_SetMotorState(MOTOR_RAMP_DOWN);
132     } else {
133         currentSpeed = conditioned;
134         if (currentSpeed < 1.0f) {
135             ActuatorData_SetMotorState(MOTOR_IDLE);
136         } else {
137             ActuatorData_SetMotorState(MOTOR_RUNNING);
138         }
139     }
140     ActuatorData_SetCurrentSpeed(currentSpeed);
141
142     /* 5. Alert evaluation */
143     AlertState_t alert = EvaluateAlert(conditioned);
144     ActuatorData_SetAlert(alert);
145
146     /* LED feedback */
147     if (alert != ALERT_NONE) {
148         LedDriver_GreenOff();
149         LedDriver_RedOn();
150     } else {
151         LedDriver_RedOff();
152         LedDriver_GreenOn();
153     }
154
155     vTaskDelayUntil(&xLastWakeTime, pdMS_TO_TICKS(
CONDITIONING_PERIOD_MS));
156 }
157 }

```

Listing 19: TaskConditioning.cpp - Conditioning Task Implementation

```
1 #pragma once
2 #include <freertos/FreeRTOS.h>
3 #include <freertos/task.h>
4
5 void TaskActuator_Task(void *pvParameters);
```

Listing 20: TaskActuator.h - Actuator Task Header

```
1 #include "TaskActuator.h"
2 #include "ActuatorData.h"
3 #include "MotorDriver.h"
4 #include "RelayDriver.h"
5
6 void TaskActuator_Task(void *pvParameters)
7 {
8     TickType_t xLastWakeTime = xTaskGetTickCount();
9
10    for (;;) {
11        /*      Analog actuator (motor/LED PWM)      */
12        float currentSpeed = ActuatorData_GetCurrentSpeed();
13        Direction_t dir     = ActuatorData_GetDirection();
14
15        /* Map 0-100% to 0-255 duty cycle */
16        uint8_t duty = (uint8_t)((currentSpeed * 255.0f) / 100.0f);
17
18        if (currentSpeed < 1.0f) {
19            MotorDriver_Stop();
20        } else {
21            MotorDriver_SetDirection((uint8_t)dir);
22            MotorDriver_SetSpeed(duty);
23        }
24
25        /*      Binary actuator (relay)      */
26        RelayState_t relayTarget = ActuatorData_GetRelayTarget();
27        RelayState_t relayCurrent = ActuatorData_GetRelayState();
28
29        if (relayTarget != relayCurrent) {
30            if (relayTarget == RELAY_ON) {
31                RelayDriver_On();
32            } else {
33                RelayDriver_Off();
34            }
35            ActuatorData_SetRelayState(relayTarget);
36        }
37    }
```

```

38     vTaskDelayUntil(&xLastWakeTime, pdMS_TO_TICKS(
        ACTUATOR_PERIOD_MS));
39 }
40 }

```

Listing 21: TaskActuator.cpp - Actuator Task Implementation

```

1  #pragma once
2  #include <freertos/FreeRTOS.h>
3  #include <freertos/task.h>
4
5  void TaskReporter_Task(void *pvParameters);

```

Listing 22: TaskReporter.h - Reporter Task Header

```

1  #include "TaskReporter.h"
2  #include "OledDisplay.h"
3  #include "ActuatorData.h"
4  #include <Arduino.h>
5  #include <stdio.h>
6
7  static const char *MotorStateName(MotorState_t st)
8  {
9      switch (st) {
10         case MOTOR_IDLE:      return "IDLE";
11         case MOTOR_RAMP_UP:    return "RAMP_UP";
12         case MOTOR_RUNNING:    return "RUNNING";
13         case MOTOR_RAMP_DOWN:  return "RAMP_DN";
14         default:               return "???";
15     }
16 }
17
18 static const char *AlertName(AlertState_t a)
19 {
20     switch (a) {
21         case ALERT_NONE:      return "NONE";
22         case ALERT_LIMIT_REACHED: return "LIMIT";
23         case ALERT_OVERLOAD:  return "OVERLD";
24         default:               return "???";
25     }
26 }
27
28 void TaskReporter_Task(void *pvParameters)
29 {
30     TickType_t xLastWakeTime = xTaskGetTickCount();
31     uint32_t reportCounter = 0;

```

```
32
33     for (;;) {
34         int16_t      target = ActuatorData_GetTargetSpeed();
35         float        cond   = ActuatorData_GetConditionedSpeed();
36         float        curr   = ActuatorData_GetCurrentSpeed();
37         Direction_t  dir    = ActuatorData_GetDirection();
38         MotorState_t mst    = ActuatorData_GetMotorState();
39         RelayState_t relay   = ActuatorData_GetRelayState();
40         AlertState_t alert   = ActuatorData_GetAlert();
41
42         /* ===== OLED 128x64 (8 lines x 21 chars at size 1) ===== */
43         char buf[22];
44         OledDisplay_Clear();
45         OledDisplay_SetTextSize(1);
46
47         OledDisplay_PrintAt(4, 0, "-- Actuator Ctrl --");
48
49         snprintf(buf, sizeof(buf), "Motor: %-7s %s",
50                 MotorStateName(mst),
51                 dir == DIR_FORWARD ? "FWD" : "REV");
52         OledDisplay_PrintAt(0, 10, buf);
53
54         snprintf(buf, sizeof(buf), "Target:  %d%%", target);
55         OledDisplay_PrintAt(0, 20, buf);
56
57         snprintf(buf, sizeof(buf), "Cond:      %.0f%%", cond);
58         OledDisplay_PrintAt(0, 30, buf);
59
60         snprintf(buf, sizeof(buf), "Current:  %.0f%%", curr);
61         OledDisplay_PrintAt(0, 40, buf);
62
63         snprintf(buf, sizeof(buf), "Relay: %s Alrt:%s",
64                 relay == RELAY_ON ? "ON " : "OFF",
65                 AlertName(alert));
66         OledDisplay_PrintAt(0, 50, buf);
67
68         snprintf(buf, sizeof(buf), "OVL:%d Ramp:%d/cyc",
69                 OVERLOAD_THRESHOLD, RAMP_STEP);
70         OledDisplay_PrintAt(0, 58, buf);
71
72         OledDisplay_Update();
73
74         /* ===== Serial Plotter (>name:value format) ===== */
75         Serial.print(">");
76         Serial.print("target:");
77         Serial.print(target);
```

```

78     Serial.print(",cond:");
79     Serial.print(cond, 1);
80     Serial.print(",current:");
81     Serial.print(curr, 1);
82     Serial.print(",relay:");
83     Serial.print(relay == RELAY_ON ? 1 : 0);
84     Serial.println();
85
86     /* ===== Structured report (every FULL_REPORT_EVERY cycles =
87     5s) ===== */
88     if ((reportCounter++ % FULL_REPORT_EVERY) == 0) {
89         printf("\n===== ACTUATOR REPORT (t=%lu ms) =====\n",
90             (unsigned long)(xTaskGetTickCount() *
portTICK_PERIOD_MS));
91         printf("Target speed   : %d%%\n", target);
92         printf("Conditioned    : %.1f%% (median=%d, EMA alpha
=%.2f)\n",
93             cond, MEDIAN_WINDOW, EMA_ALPHA);
94         printf("Current speed : %.1f%% (ramp step=%d/cyc)\n",
95             curr, RAMP_STEP);
96         printf("Direction     : %s\n",
97             dir == DIR_FORWARD ? "FORWARD" : "REVERSE");
98         printf("Motor state    : %s\n", MotorStateName(mst));
99         printf("Relay         : %s\n",
100             relay == RELAY_ON ? "ON" : "OFF");
101         printf("Alert         : %s (overload >= %d%%)\n",
102             AlertName(alert), OVERLOAD_THRESHOLD);
103         printf("PWM duty      : %d/255\n",
104             (int)((curr * 255.0f) / 100.0f));
105         printf("=====\n\n");
106     }
107     vTaskDelayUntil(&xLastWakeTime, pdMS_TO_TICKS(
REPORTER_PERIOD_MS));
108 }
109 }

```

Listing 23: TaskReporter.cpp - Reporter Task Implementation